

Cloud & Data Engineering - Introduction

IA Institut - 2025/2026

Section 0

Internship

Section 1

Real World Settings

A DAY IN DATA

The exponential growth of data is undisputed, but the numbers behind this explosion – fuelled by internet of things and the use of connected devices – are hard to comprehend, particularly when looked at in the context of one day

500m

tweets are sent every day

Twitter



4PB

of data created by Facebook, including

350m photos

100m hours of video watch time

Facebook Research

294bn

billion emails are sent

Radicati Group

320bn

emails to be sent each day by 2021

306bn

emails to be sent each day by 2020

3.9bn

people use emails

4TB

of data produced by a connected car

Intel

ACCUMULATED DIGITAL UNIVERSE OF DATA

4.4ZB

44ZB

PwC

2013

2020

DEMYSIFYING DATA UNITS

From the more familiar 'bit' or 'megabyte', larger units of measurement are more frequently being used to explain the masses of data

Unit	Value	Size
b bit	0 or 1	1/8 of a byte
B byte	8 bits	1 byte
KB kilobyte	1,000 bytes	1,000 bytes
MB megabyte	1,000 ² bytes	1,000,000 bytes
GB gigabyte	1,000 ³ bytes	1,000,000,000 bytes
TB terabyte	1,000 ⁴ bytes	1,000,000,000,000 bytes
PB petabyte	1,000 ⁵ bytes	1,000,000,000,000,000 bytes
EB exabyte	1,000 ⁶ bytes	1,000,000,000,000,000,000 bytes
ZB zettabyte	1,000 ⁷ bytes	1,000,000,000,000,000,000,000 bytes
YB yottabyte	1,000 ⁸ bytes	1,000,000,000,000,000,000,000,000 bytes

*A lowercase "b" is used as an abbreviation for bits, while an uppercase "B" represents bytes.

65bn

messages sent over WhatsApp and two billion minutes of voice and video calls made

Facebook



Searches made a day

5bn

Searches made a day from Google

3.5bn

Smart Insights



463EB

of data will be created every day by 2025

ioe

95m

photos and videos are shared on Instagram

Instagram Business



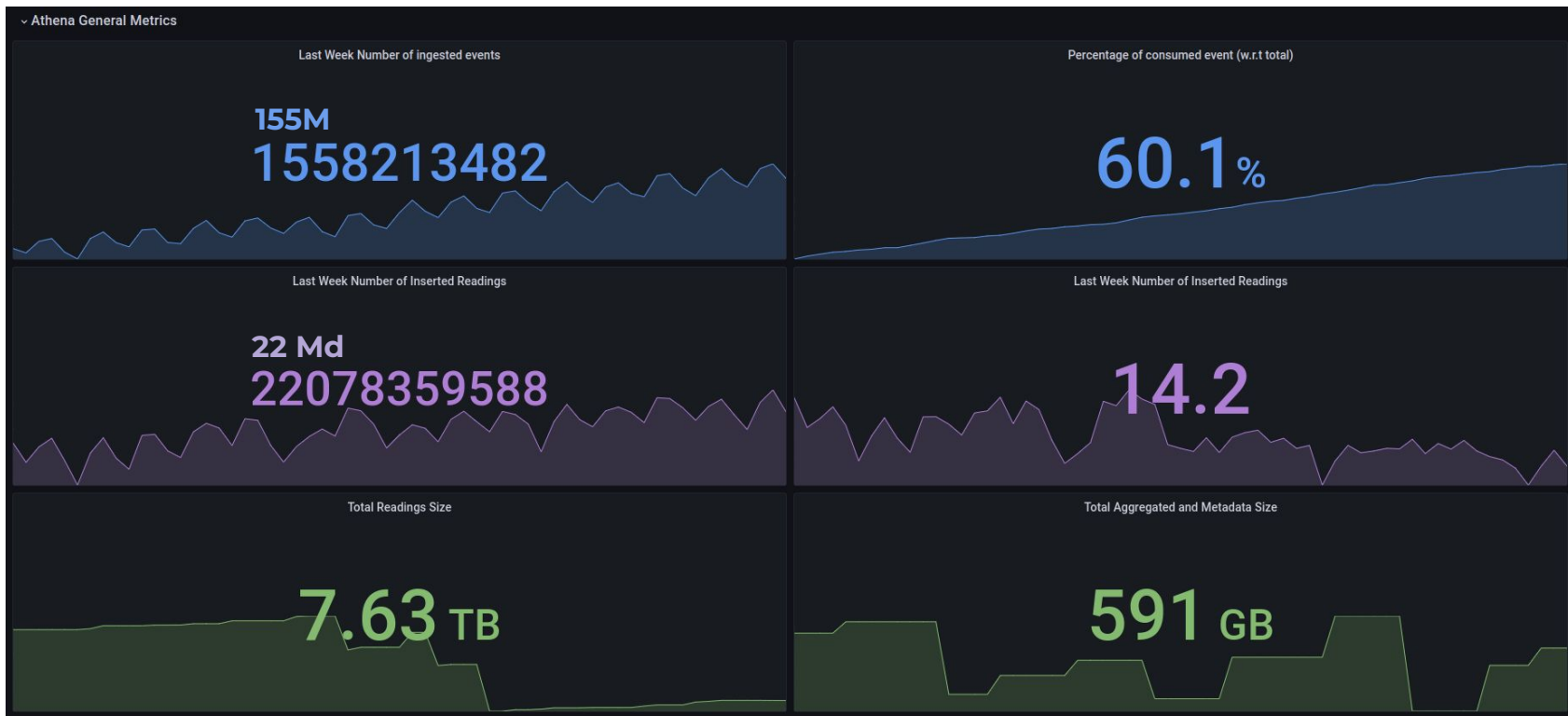
28PB

to be generated from wearable devices by 2020

Statista



Munic Use Case





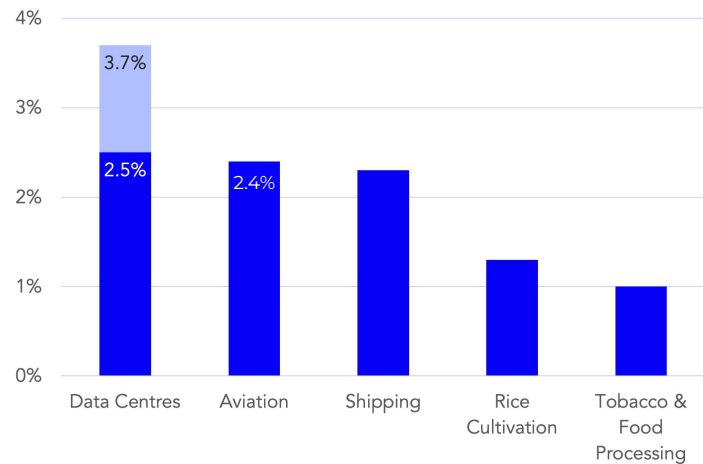
Source: <https://www.datacentermap.com/datacenters/>

2,000km



Global cloud computing emissions exceed those from commercial aviation

Share of global CO₂ emission generated by sector/category



Source: Climatiq Analysis, The Shift Project, OurWorldinData



<https://www.climatiq.io/blog/measure-greenhouse-gas-emissions-carbon-data-centres-cloud-computing> (2022)

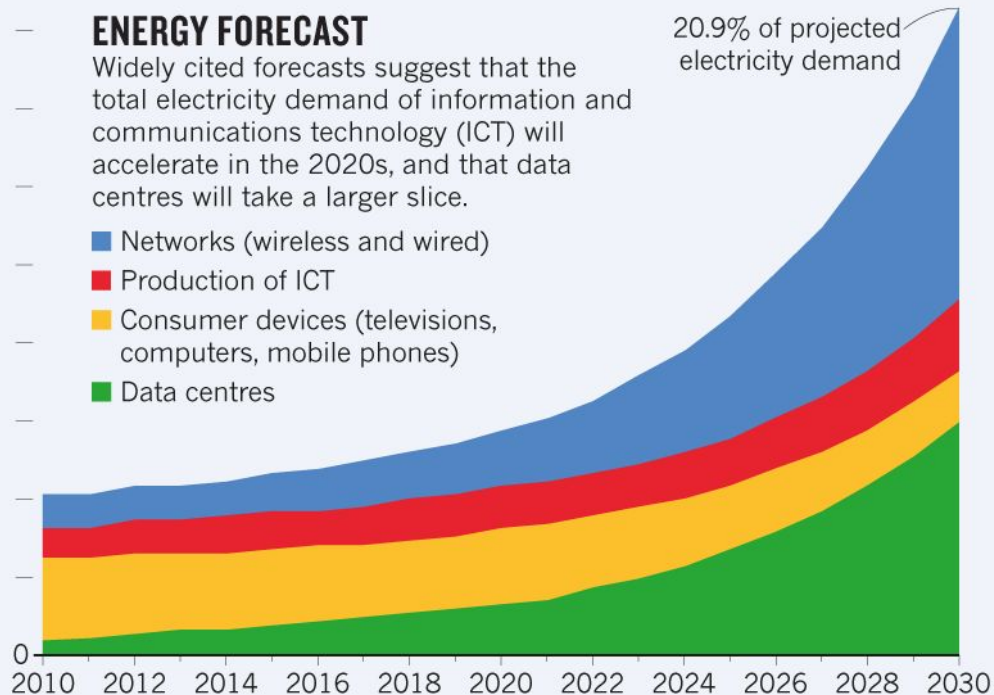
* <https://theshiftproject.org/en/article/lean-ict-our-new-report/>

9,000 terawatt hours (TWh)

ENERGY FORECAST

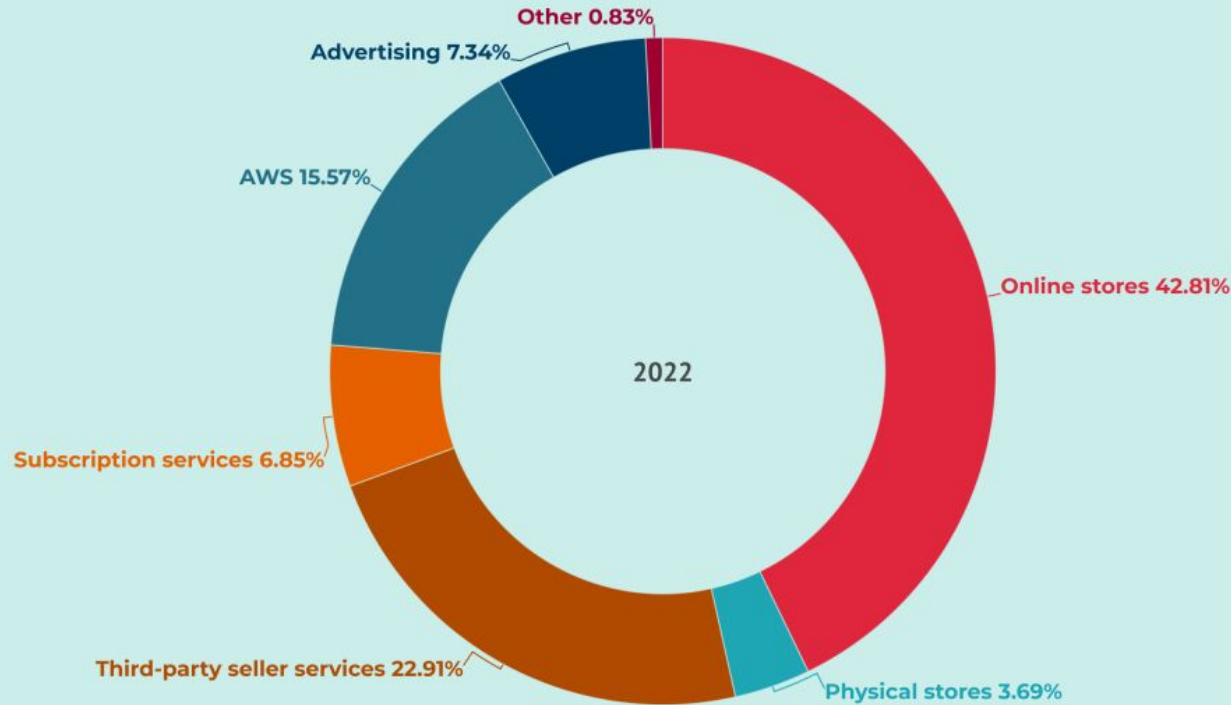
Widely cited forecasts suggest that the total electricity demand of information and communications technology (ICT) will accelerate in the 2020s, and that data centres will take a larger slice.

- Networks (wireless and wired)
- Production of ICT
- Consumer devices (televisions, computers, mobile phones)
- Data centres



<https://www.akcp.com/blog/the-real-amount-of-energy-a-data-center-use/>

Amazon Revenue Breakdown



80
Billions \$

Cloud Computing as
an important source
of revenue



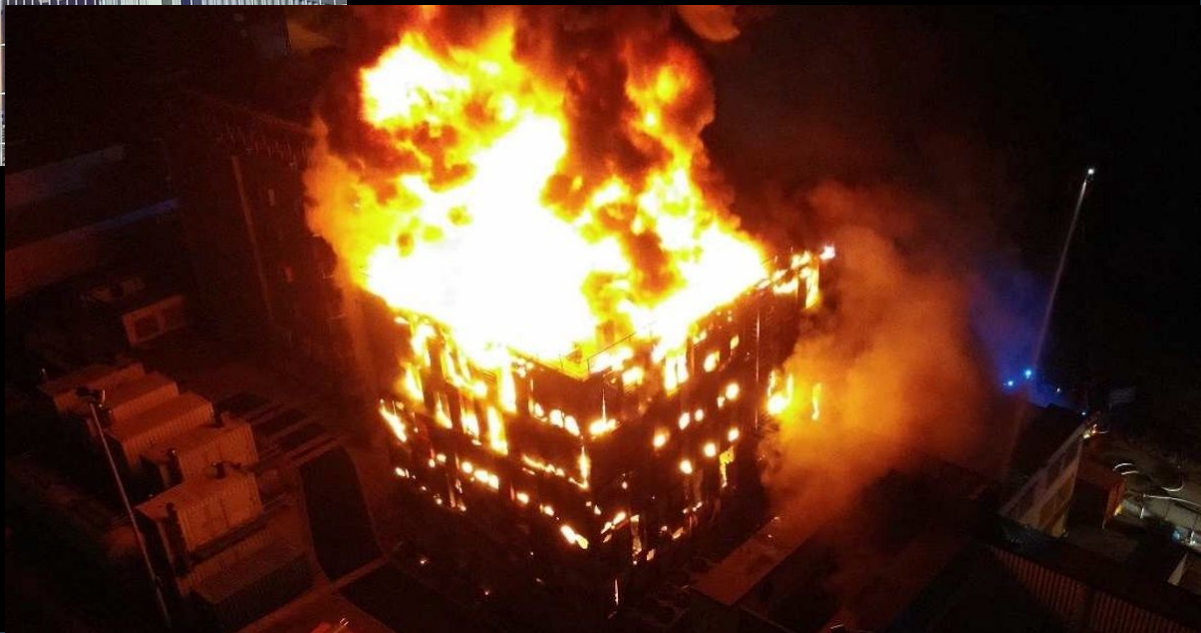
Source:

<https://web.developpez.com/actu/318808/Un-incendie-pourrait-couter-105-millions-d-euros-a-OVHcloud-selon-les-documents-relatifs-a-l-introduction-en-bourse/>

Source:

<https://www.solutions-numeriques.com/exclusif-les-causes-de-lincendie-ovh-strasbourg-des-exlications-stupefiantes/>

3,6 millions de sites internet affectés



Section 2

Words that should be understood after this Curriculum

Notions à Apprendre

IAAS

Architecture

Backend

Broker

Docker

Scaleway

minikube

Virtualisation

SAAS

Kubernetes

PAAS

Kafka

API

AWS

GCP

OVH

Frontend

Cloud

Vagrant

Conteneurisation

Docker
Compose

Azure

Section 3

Cloud Computing



**What do you
understand by the
term “Cloud” ?**



Who coined the term “Cloud Computing” ?

What's interesting [now] is that there is an **emergent new model**, and you all are here because you are part of that new model. I don't think people have really understood how big this opportunity really is. It starts with the premise that the **data services** and **architecture** should be on servers. We call it **cloud computing** – they should be in a “**cloud**” somewhere. And that if you have the right kind of browser or the right kind of access, it doesn't matter whether you have a PC or a Mac or a mobile phone or a BlackBerry or what have you – or new devices still to be developed – you can get access to the cloud. There are a number of companies that have benefited from that. ...

Eric Schmidt (CEO of Google) - 2006



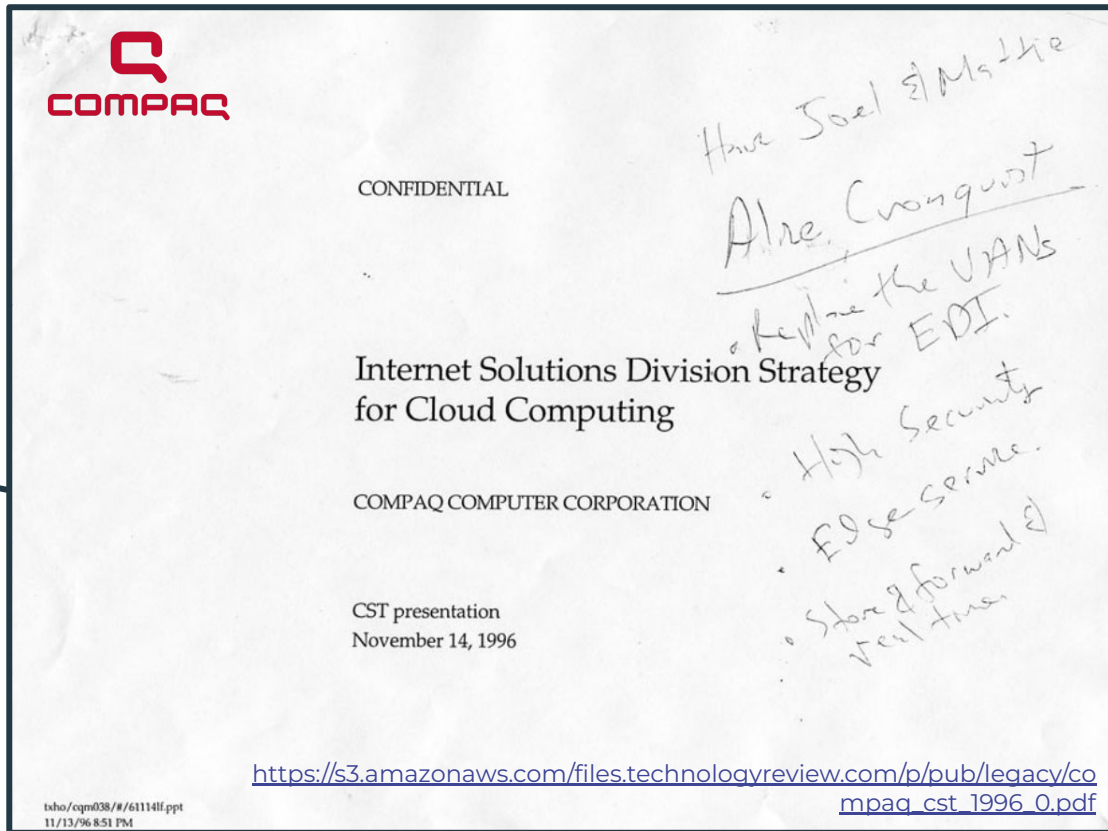
The screenshot shows a Google Press Center page. At the top, there's a search bar and navigation links for 'About Google', 'News from Google', 'Podium Archive', and 'Search Engine Strategies Conference'. The main content is dated 'August 9, 2006' and titled 'Search Engine Strategies Conference' and 'Conversation with Eric Schmidt hosted by Danny Sullivan'. The transcript shows a dialogue between Danny and Eric. Danny starts by welcoming everyone and mentioning the Google Dance. Eric thanks him. Danny then introduces the topic of cloud computing, mentioning the 'data services' and 'architecture' being on servers. Eric responds by saying that the business model was largely invented by Oracle and that the term 'cloud computing' was coined by him and others at Google, Yahoo!, eBay, Amazon, etc. He also mentions that the term 'cloud' was used by him and others at Google, Yahoo!, eBay, Amazon, etc. The transcript ends with a link to the full transcript: <https://www.google.com/press/podium/ses2006.html>

Who coined the term “Cloud Computing” ?

George Favaloro holds Compaq's 1996 Business Plan. This is the earliest printed document that mentions the term **“cloud computing”**.



<https://prog.world/who-coined-the-term-cloud-computing/>



Cloud - Definitions

Cloud computing is the **on-demand availability of computer system resources**, especially data storage (cloud storage) and computing power, without direct active management by the user.

https://en.wikipedia.org/wiki/Cloud_computing

Cloud - Definitions

Cloud computing is the **on-demand availability of computer system resources**, especially data storage (cloud storage) and computing power, without direct active management by the user.

https://en.wikipedia.org/wiki/Cloud_computing

(4) **Cloud.** a network of **servers** (= computers that control or supply information to other computers) on which **data** and **software** can be stored or managed and to which **users have access over the internet**

<https://www.oxfordlearnersdictionaries.com/>

Cloud - Definitions

Cloud computing is the **on-demand availability of computer system resources**, especially data storage (cloud storage) and computing power, without direct active management by the user.

https://en.wikipedia.org/wiki/Cloud_computing

(4) **Cloud.** a network of **servers** (= computers that control or supply information to other computers) on which **data** and **software** can be stored or managed and to which **users have access over the internet**

<https://www.oxfordlearnersdictionaries.com/>

Le **cloud computing** (en français, « **informatique dans les nuages** ») fait référence à l'utilisation de la mémoire et des capacités de calcul **des ordinateurs et des serveurs répartis dans le monde entier et liés par un réseau.**

<https://www.cnil.fr/fr/definition/cloud-computing>

Cloud - Definitions

Cloud computing is the **on-demand availability of computer system resources**, especially data storage (cloud storage) and computing power, without direct active management by the user.

https://en.wikipedia.org/wiki/Cloud_computing

(4) **Cloud.** a network of **servers** (= computers that control or supply information to other computers) on which **data** and **software** can be stored or managed and to which **users have access over the internet**

<https://www.oxfordlearnersdictionaries.com/>

Le **cloud computing** (en français, « **informatique dans les nuages** ») fait référence à l'utilisation de la mémoire et des capacités de calcul **des ordinateurs et des serveurs répartis dans le monde entier et liés par un réseau.**

<https://www.cnil.fr/fr/definition/cloud-computing>

Cloud computing is the increasingly common practice of providing **'on-demand'** cloud services as IT infrastructure and resources.

<https://www.ovhcloud.com/en/public-cloud/cloud-computing/>

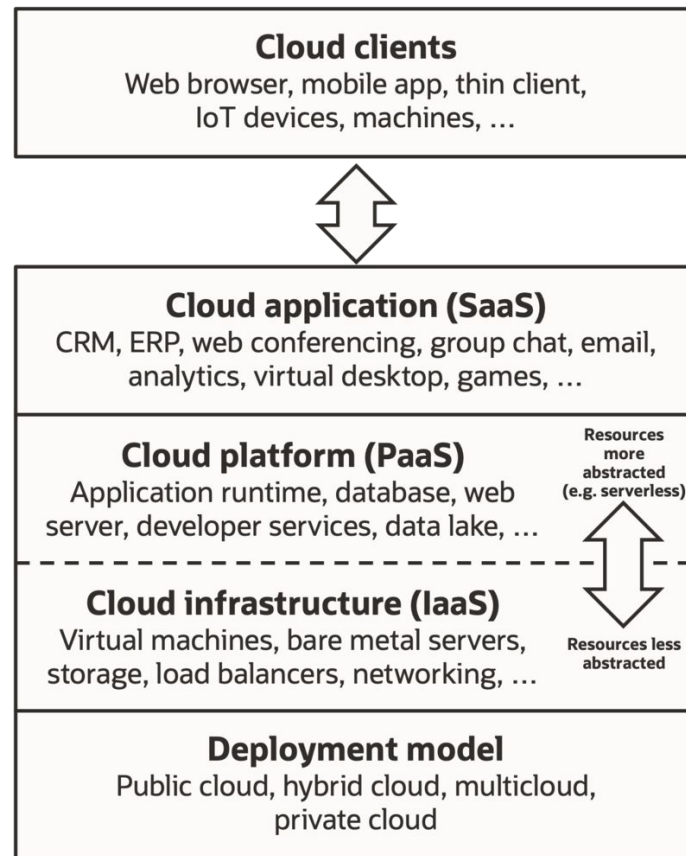
Cloud - Definitions

Cloud computing is a model for enabling **ubiquitous, convenient, on-demand** network access to a **shared pool of configurable computing resources** (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with **minimal management effort** or service provider interaction.

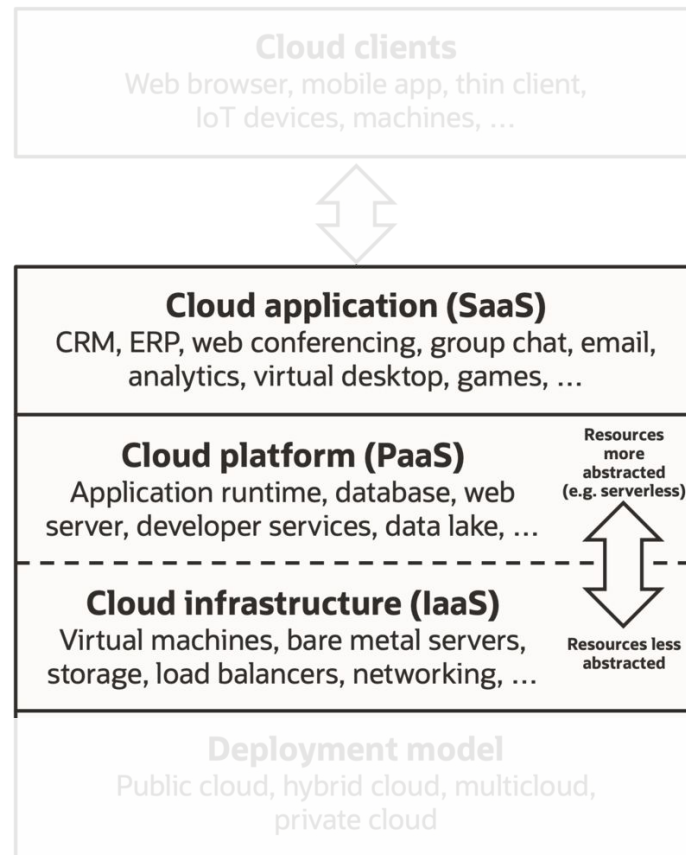
<https://nvlpubs.nist.gov/nistpubs/legacy/sp/nistspecialpublication800-145.pdf>

(NIST - National Institute of Standards and Technology - 2011)

Cloud - Service Models



Cloud - Service Models

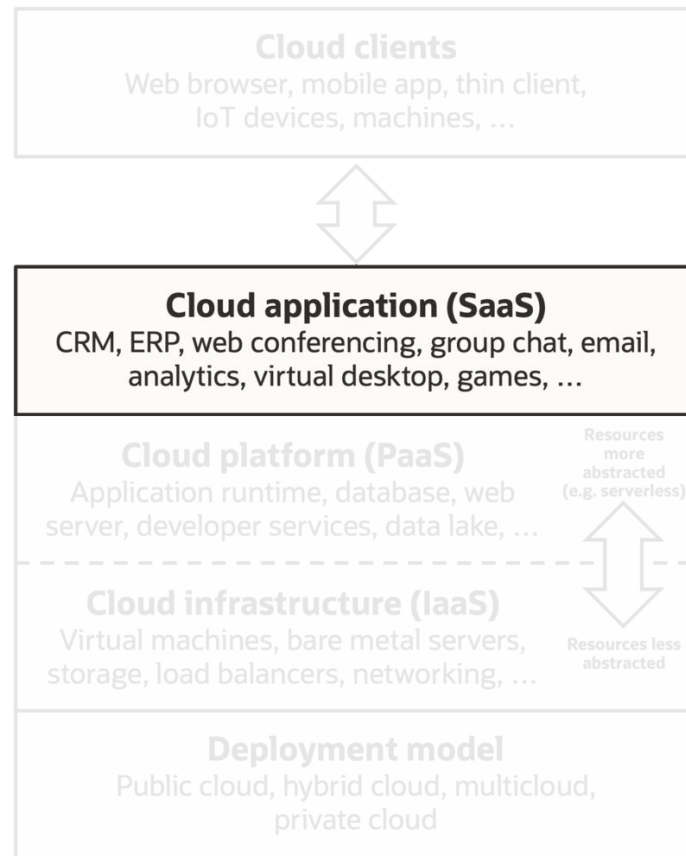


Cloud - Provided Services

Software as as Service (SaaS)

The capability provided to the consumer is to use the provider's applications running on a cloud infrastructure . The applications are accessible from various client devices [...]. The consumer does not manage or control the underlying cloud infrastructure including network, servers, operating systems, storage, or even individual application capabilities, with the possible exception of limited user specific application configuration settings.

Examples: Gmail, Slack, Google Drive, etc.



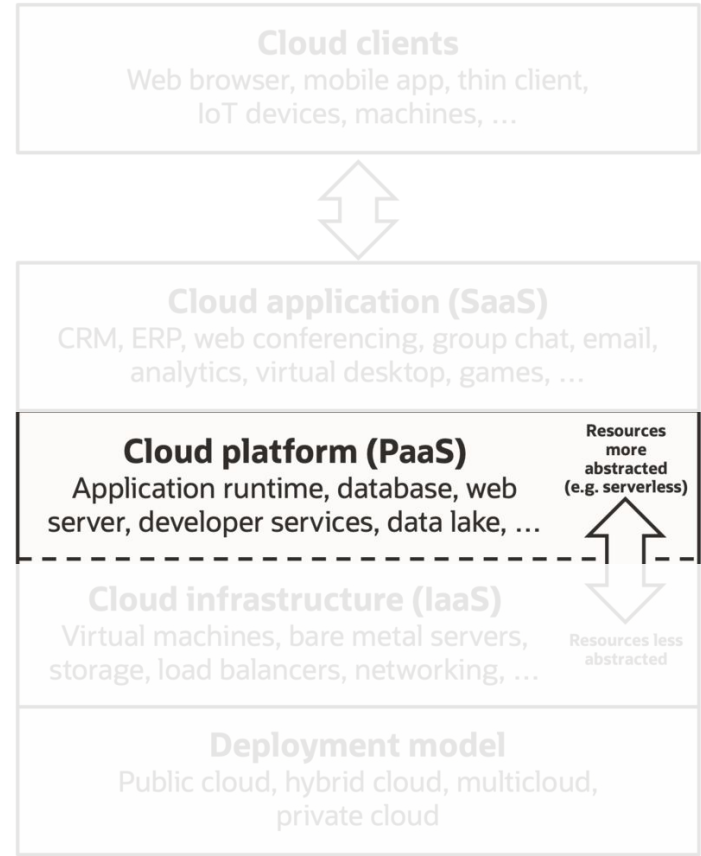
Cloud - Provided Services

Platform as a Service (PaaS)

Platform as a Service (PaaS). The capability provided to the consumer is to deploy onto the cloud infrastructure consumer-created or acquired applications created using programming languages, libraries, services, and tools supported by the provider. The consumer does not manage or control the underlying cloud infrastructure including network, servers, operating systems, or storage, but has control over the deployed applications and possibly configuration settings for the application-hosting environment.

Examples: Heroku, Google App Engine, Amazon Lambda (FaaS - Function as a Service), ...

<https://nvlpubs.nist.gov/nistpubs/legacy/sp/nistspecialpublication800-145.pdf>



https://en.wikipedia.org/wiki/Cloud_computing

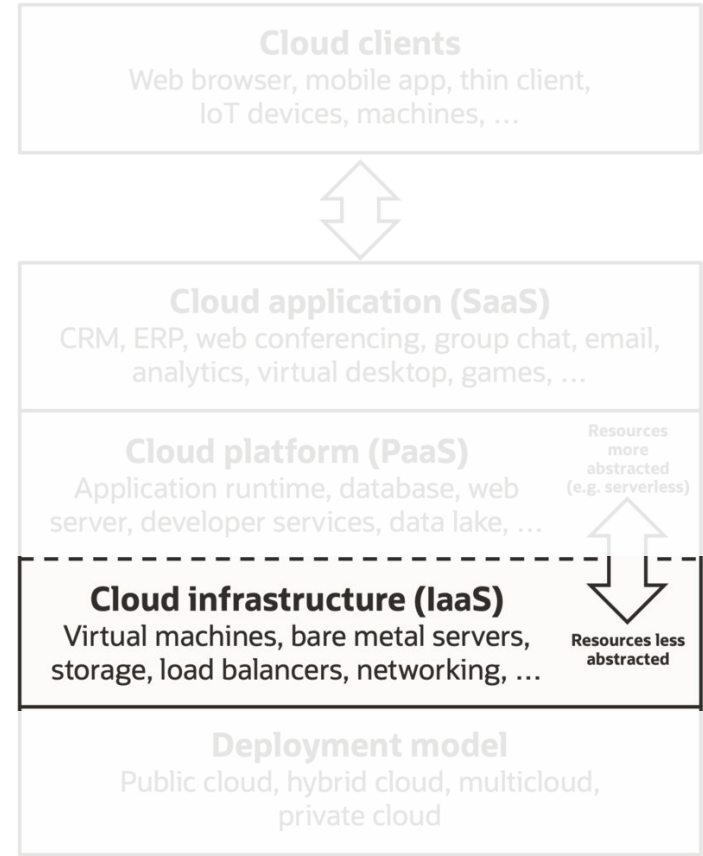
Cloud - Provided Services

Infrastructure as as Service (IaaS)

The capability provided to the consumer is to provision processing, storage, networks, and other fundamental computing resources where the consumer is able to deploy and run arbitrary software, which can include operating systems and applications. The consumer does not manage or control the underlying cloud infrastructure but has control over operating systems, storage, and deployed applications; and possibly limited control of select networking components (e.g., host firewalls).

Examples: Google Cloud Platform (GCP), Amazon Web Services (AWS), Google Azure, OVH Public Cloud, ...

<https://nvlpubs.nist.gov/nistpubs/legacy/sp/nistspecialpublication800-145.pdf>



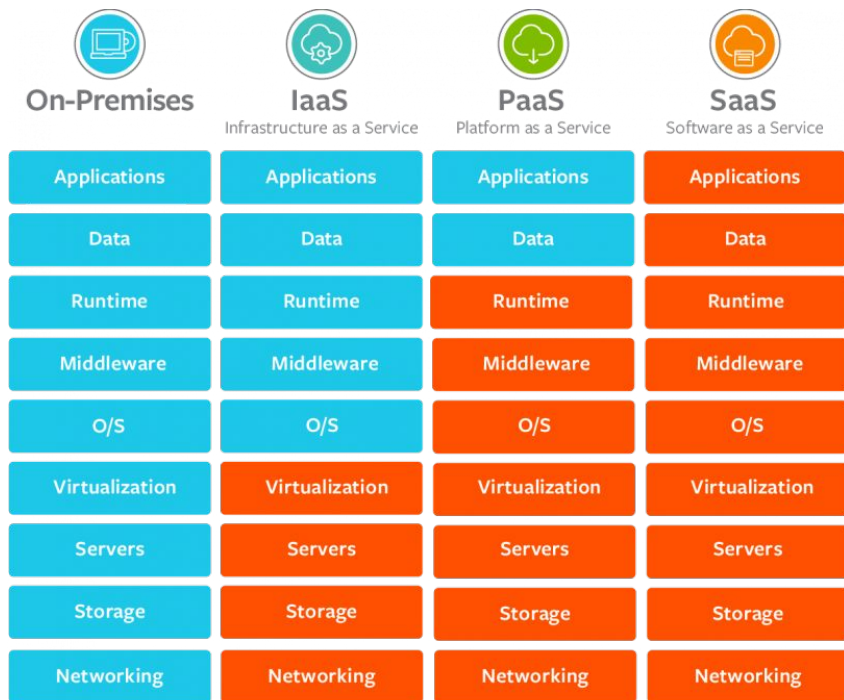
https://en.wikipedia.org/wiki/Cloud_computing

Cloud - Cloud Computing vs On-Premise

On-Premise

On-premises refers to IT infrastructure hardware and software applications that are **hosted on-site**. This contrasts with IT assets that are hosted by a public cloud platform or remote data center.

https://www.insight.com/en_US/content-and-resources/glossary/o/on-premises.html



<https://www.bmc.com/blogs/saas-vs-paas-vs-iaas-whats-the-difference-and-how-to-choose/>

Cloud - Deployment Models



Public clouds

A cloud environment created from resources not owned by the end user that can be redistributed to other tenants.



Private clouds

Loosely defined as a cloud environment solely dedicated to the end user, usually within the user's firewall and sometimes on premise.



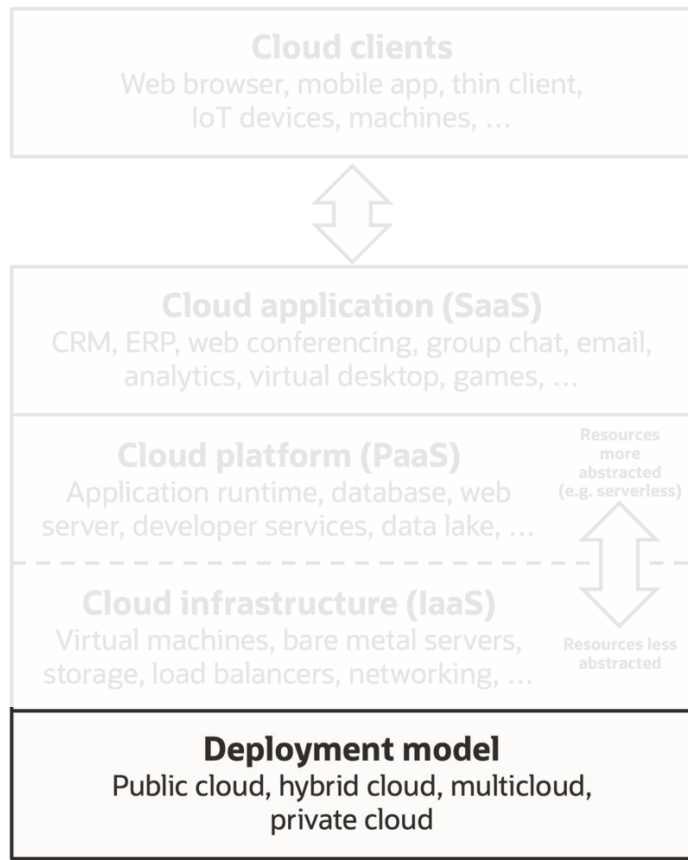
Hybrid clouds

Multiple cloud environments with some degree of workload portability, orchestration, and management among them.



Multiclouds

An IT system that includes more than 1 cloud—public or private—that may or may not be networked together.



Cloud - Deployment Models



Public clouds

A cloud environment created from resources not owned by the end user that can be redistributed to other tenants.



Private clouds

Loosely defined as a cloud environment solely dedicated to the end user, usually within the user's firewall and sometimes on premise.



Hybrid clouds

Multiple cloud environments with some degree of workload portability, orchestration, and management among them.



Multiclouds

An IT system that includes more than 1 cloud—public or private—that may or may not be networked together.

Cloud - Deployment Models

Public Clouds

A public cloud is a pool of virtual resources—developed from hardware owned and managed by a third-party company—that is automatically provisioned and allocated among multiple clients through a self-service interface.

Examples: Google Cloud Platform (GCP), Amazon Web Services (AWS), Google Azure, OVH Public Cloud, ...



Public clouds

A cloud environment created from resources not owned by the end user that can be redistributed to other tenants.



Private clouds

Loosely defined as a cloud environment solely dedicated to the end user, usually within the user's firewall and sometimes on premise.



Hybrid clouds

Multiple cloud environments with some degree of workload portability, orchestration, and management among them.



Multiclouds

An IT system that includes more than 1 cloud—public or private—that may or may not be networked together.

Cloud - Deployment Models

Private Clouds

Private Cloud are loosely defined as cloud environments solely dedicated to a single end user or group.

Private clouds no longer have to be sourced from **on-premise IT infrastructure**.

Organizations are now building private clouds on rented, vendor-owned data centers located off-premises. This has also led to a number of private cloud subtypes, including **managed private clouds** and **dedicated clouds**.

Examples: OVH Hosted Private Cloud.



Public clouds

A cloud environment created from resources not owned by the end user that can be redistributed to other tenants.



Private clouds

Loosely defined as a cloud environment solely dedicated to the end user, usually within the user's firewall and sometimes on premise.



Hybrid clouds

Multiple cloud environments with some degree of workload portability, orchestration, and management among them.



Multiclouds

An IT system that includes more than 1 cloud—public or private—that may or may not be networked together.

Cloud - Deployment Models

Multiclouds and Hybrid Clouds

Multiclouds are a cloud approach made up of more than 1 cloud service, from **more than 1 cloud vendor** - public or private. All hybrid clouds are multiclouds, but not all multiclouds are hybrid clouds. Multiclouds become **hybrid clouds** when **multiple clouds are connected by some form of integration or orchestration**.



Public clouds

A cloud environment created from resources not owned by the end user that can be redistributed to other tenants.



Private clouds

Loosely defined as a cloud environment solely dedicated to the end user, usually within the user's firewall and sometimes on premise.



Hybrid clouds

Multiple cloud environments with some degree of workload portability, orchestration, and management among them.



Multiclouds

An IT system that includes more than 1 cloud—public or private—that may or may not be networked together.

Section 3

Cloud and Data Engineering

Cloud & Data Engineering

Cloud & Data Engineering

Cloud & Data Engineering

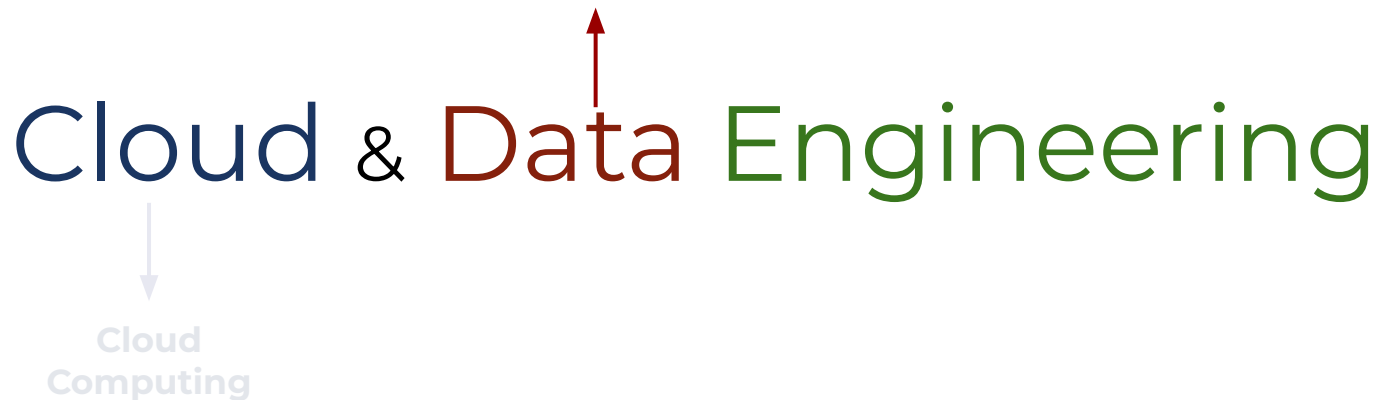
Cloud & Data Engineering



**Cloud
Computing**

Cloud & Data Engineering

Data is the physical representation of information in a manner suitable for communication, interpretation, or processing by human beings or by **automatic means**.



Cloud & Data Engineering

Data is the physical representation of information in a manner suitable for communication, interpretation, or processing by human beings or by **automatic means**.

Cloud & Data Engineering

```
graph TD; CC[Cloud Computing] --> CDE[Cloud & Data Engineering]; DE[Data Engineering] --> CDE;
```

Cloud
Computing

* The **creative application of scientific principles** to **design** or **develop structures**, machines, apparatus, or manufacturing processes, or works utilizing them singly or in combination; or to construct or operate the same with full cognizance of their design; or to forecast their behavior under specific operating conditions; all as respects an intended function, economics of operation and safety to life and property.

Cloud & Data Engineering

Cloud Engineering

Cloud engineering is the application of **engineering disciplines** to **cloud computing**. It brings a systematic approach to concerns of commercialization, standardization, and governance of cloud computing applications. In practice, it leverages the methods and tools of engineering in **conceiving, developing, operating** and **maintaining cloud computing systems** and **solutions**.

https://en.wikipedia.org/wiki/Cloud_engineering

Data Engineering

Data engineering refers to the building of systems to enable the **collection** and **usage** of **data**. This data is usually used to enable subsequent analysis and data science; which often involves machine learning. Making the data usable usually involves substantial **compute** and **storage**, as well as **data processing**.

https://en.wikipedia.org/wiki/Data_engineering

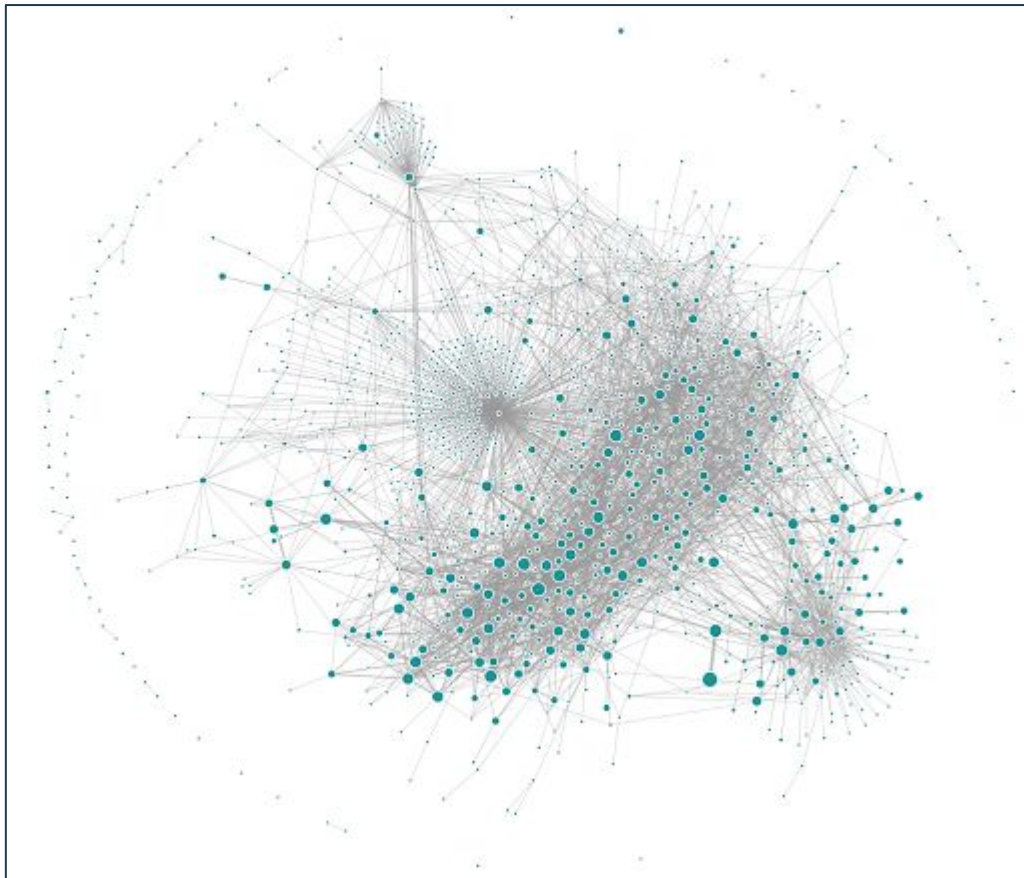
Section 4

Cloud and Architecture - Part I

Uber's microservice architecture circa mid-2018 from Jaeger*

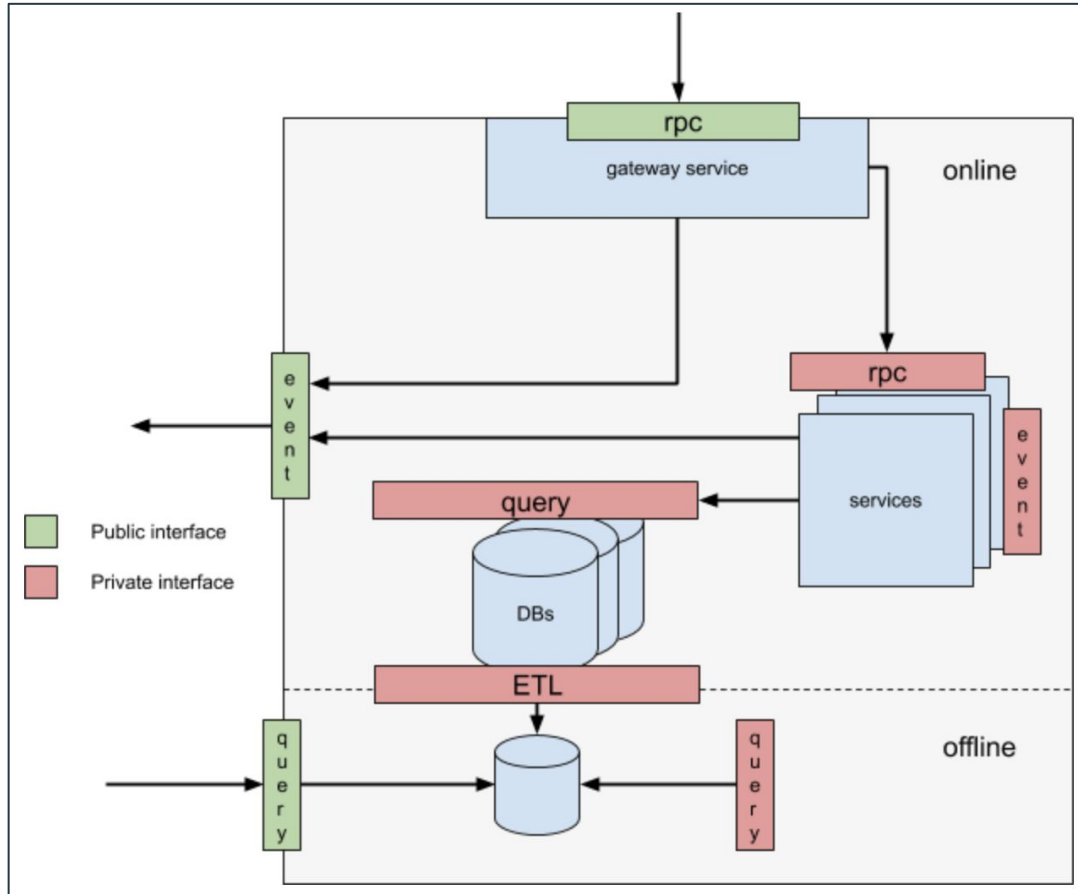


***Jaeger** is open source software for
tracing transactions between
distributed **services**

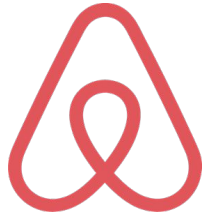


<https://www.uber.com/en-FR/blog/microservice-architecture> [2020]

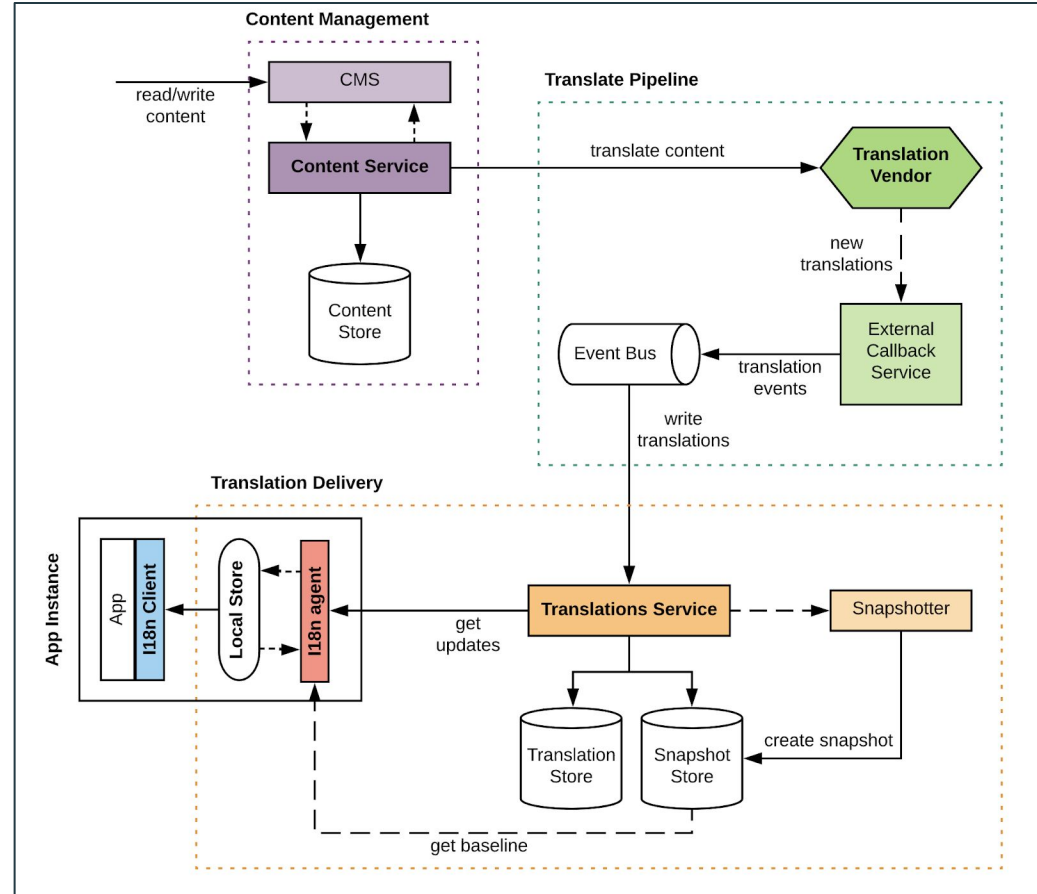
Gateway API in a Domain-Oriented Microservice Architecture (Uber)



Airbnb's Internationalization* Platform

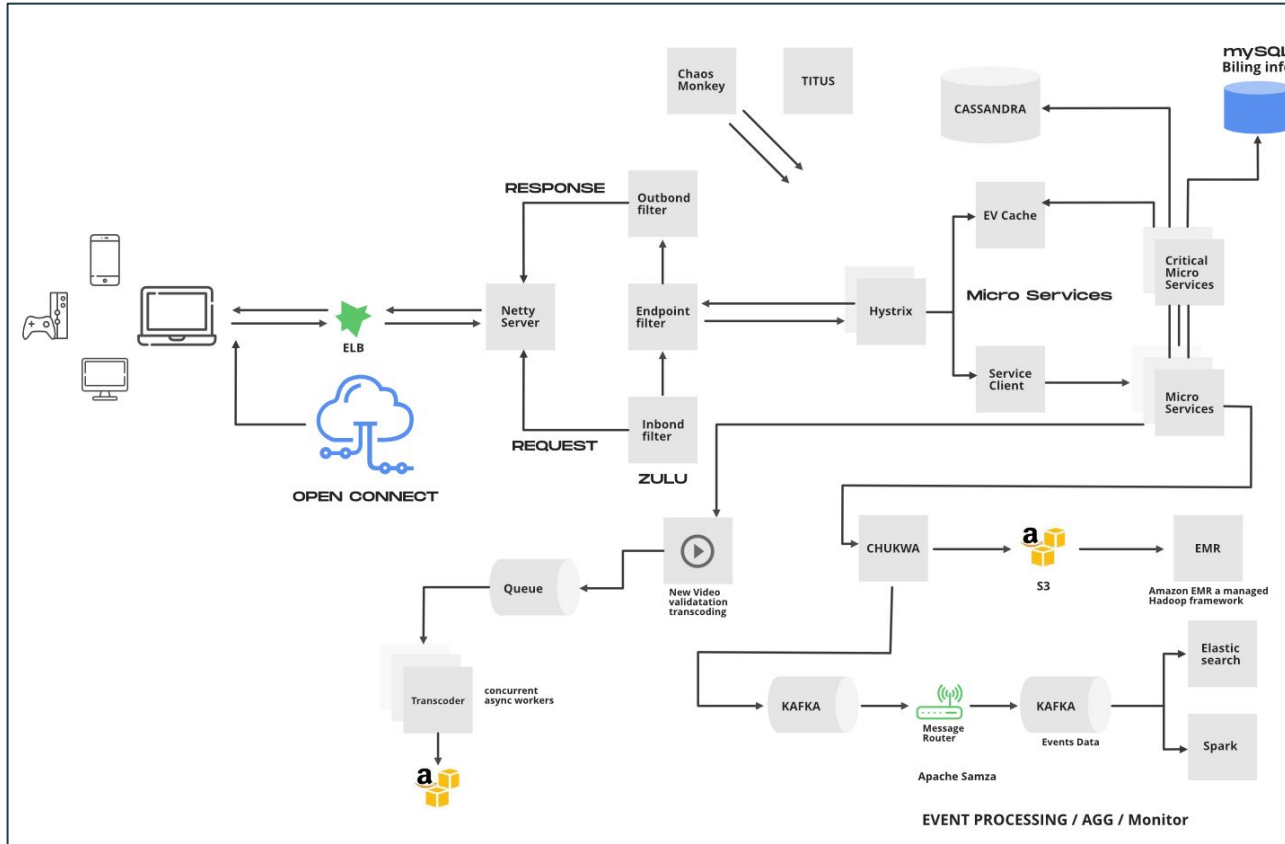


****Internationalization** is the process of adapting software to accommodate for different languages, cultures, and regions (i.e. locales), while minimizing additional engineering changes required for localization.*

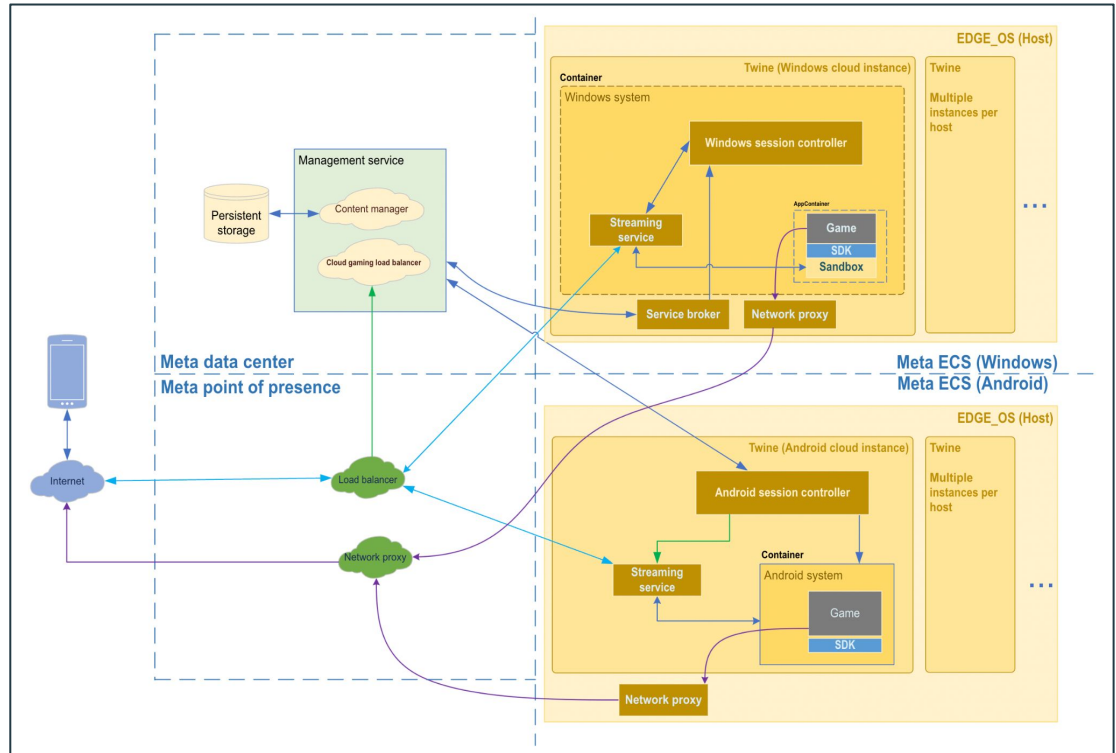
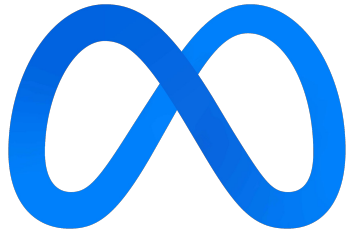


Netflix Backend Architecture

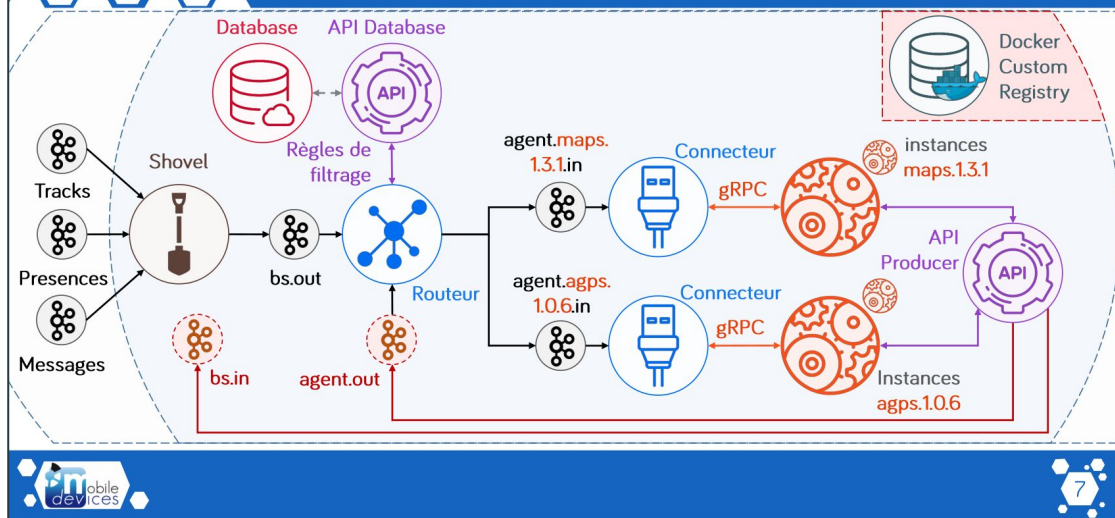
NETFLIX



Meta's Cloud Gaming Infrastructure



Architecture proposée alpha



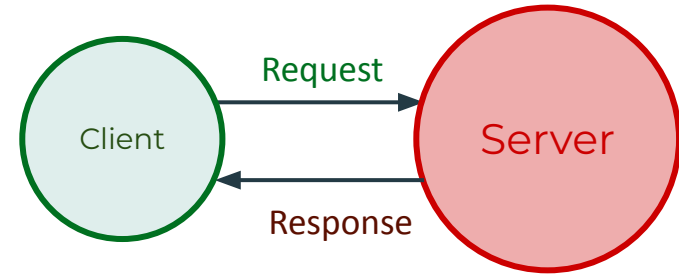
Munic - cloud V4 Draft (2016)



Architecture Patterns - Building Patterns

Client-Server (request-response) Pattern

In the client-server architecture patterns, there are two main components: The client, which is the **service requester**, and the server, which is the **service provider**. Although both client and server may be located within the same system, they often communicate over a network on separate hardware.

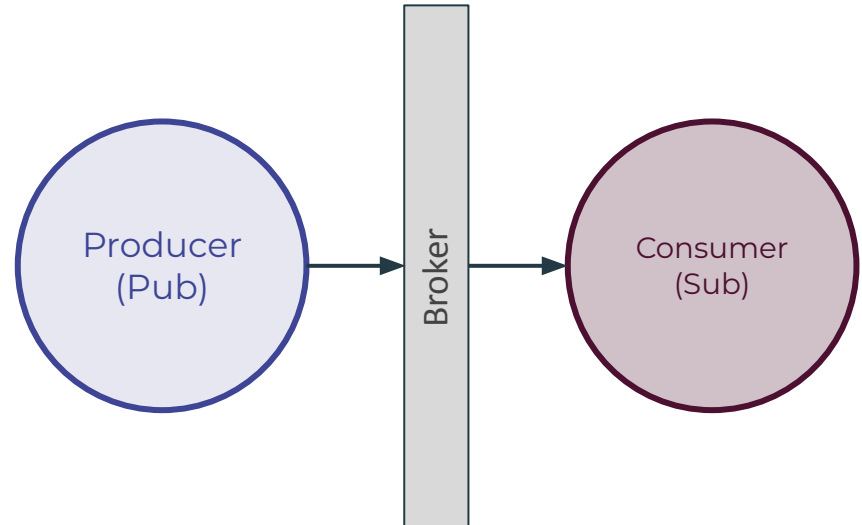


Architecture Patterns - Building Patterns

Publisher-Subscriber (Producer-Consumer) Pattern

Enable an application to **announce events** to multiple interested consumers **asynchronously***, **without coupling** the **senders (publishers)** to the **receivers (consumers)**.

* **Asynchronous communication** differs from **synchronous communication** in the way that in asynchronous communication the producer **does not need to wait** the receiver to handle the produced events/data.



<https://learn.microsoft.com/en-us/azure/architecture/patterns/publisher-subscriber>

Section 5 - Exercise

Cloud and Architecture - Part II -
Implementing a Client-Server Pattern

Architecture Patterns - Client-Server Pattern

Exercise 1:

Implement a simple **REST API** to manage products in a store. A **Product** has a **name**, **price** and **quantities** for attributes. At Least **POST** and **GET** should be implemented.



Your REST API in Python or any other chosen language

cURL linux command line tool

Architecture Patterns - Client-Server Pattern

Exercise 1:

Implement a simple **REST API** to manage products in a store. A **Product** has a **name**, **price** and **quantities** for attributes. At Least **POST** and **GET** should be implemented.

Questions:

What is an API ? Why API and not just a Web Interface ? What a REST API ?

Technologies:

Use whatever language you want. If you are using Python, we suggest **fastapi** and **pydantic**.

Warning:

For now, the data stored by your API does not need to be persistent.

```
curl -X 'POST' 'http://localhost:8000/products/' -H 'accept: application/json' -H 'Content-Type: application/json' -d '{"name": "Orange", "price": 4, "quantity": 12}'
```

```
curl -X 'GET' 'http://localhost:8000/products/Orange'
```

```
'{"name": "Orange", "price": 4, "quantity": 12}'
```

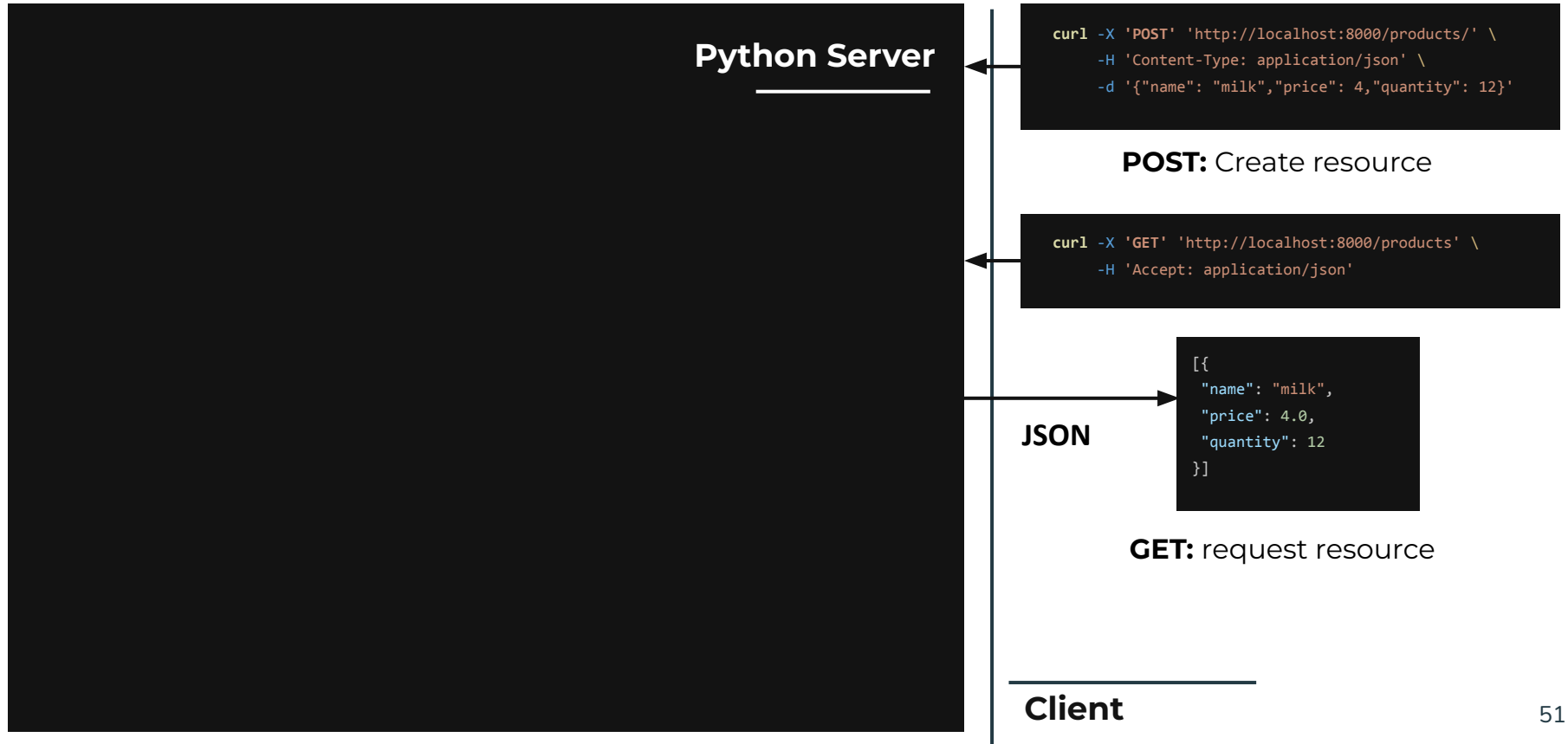
Your REST API may also be implemented in any other chosen language

CURL linux command line tool

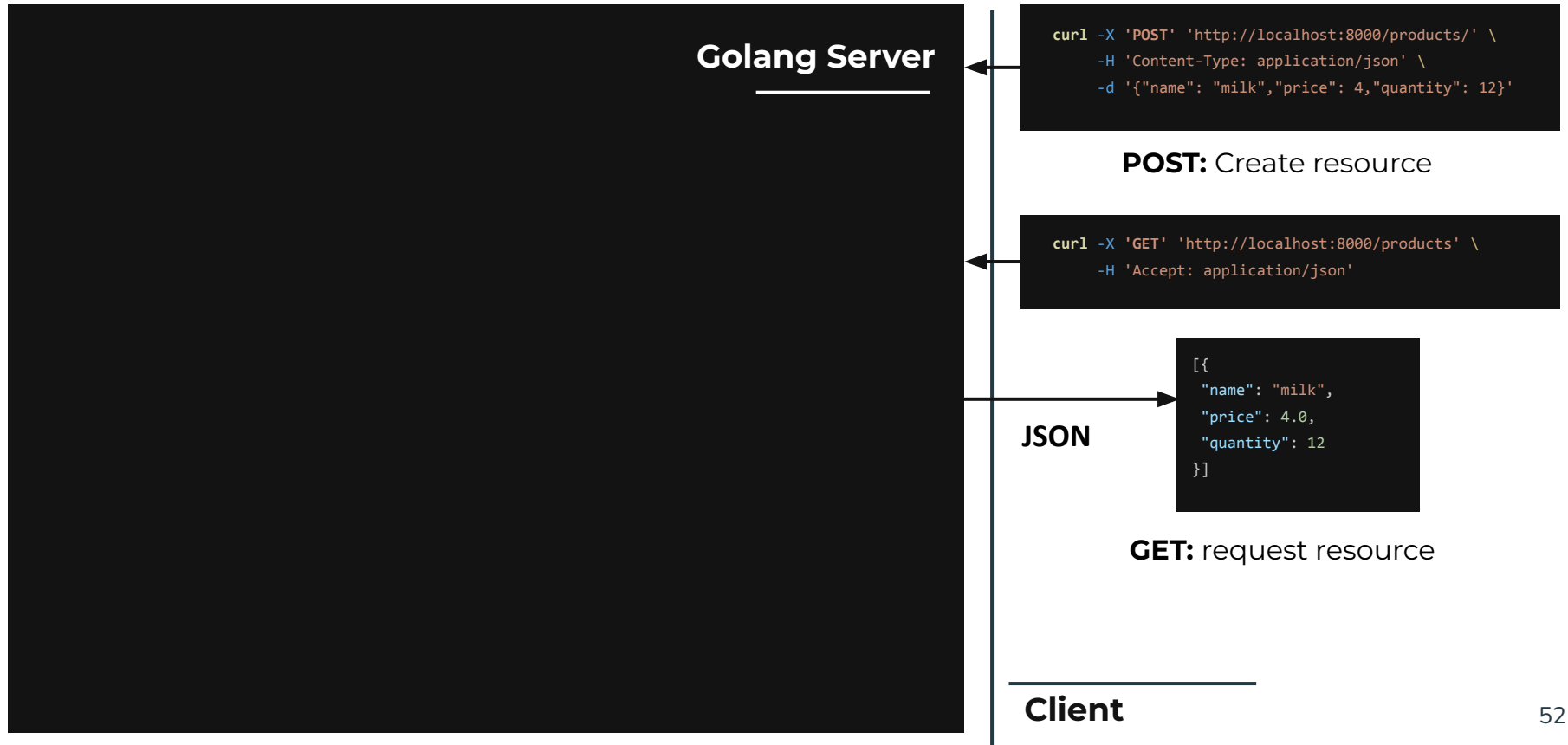
Section 5 - Solution

Cloud and Architecture - Part II -
Implementing a Client-Server Pattern

Architecture Patterns - Client-Server Pattern



Architecture Patterns - Client-Server Pattern



Architecture Patterns - Client-Server Pattern

No matter what language is used

```
curl -X 'POST' 'http://localhost:8000/products/' \  
-H 'Content-Type: application/json' \  
-d '{"name": "milk", "price": 4, "quantity": 12}'
```

POST: Create resource

```
curl -X 'GET' 'http://localhost:8000/products' \  
-H 'Accept: application/json'
```

JSON

```
[{  
  "name": "milk",  
  "price": 4.0,  
  "quantity": 12  
}]
```

GET: request resource

Client

Architecture Patterns - Client-Server Pattern

No matter what language is used

What matters here:

Communication
Protocol (set of rules):
HTTP

Data Serialization
Format:
JSON

```
curl -X 'POST' 'http://localhost:8000/products/' \  
-H 'Content-Type: application/json' \  
-d '{"name": "milk", "price": 4, "quantity": 12}'
```

POST: Create resource

```
curl -X 'GET' 'http://localhost:8000/products' \  
-H 'Accept: application/json'
```

```
[{  
  "name": "milk",  
  "price": 4.0,  
  "quantity": 12  
}]
```

JSON

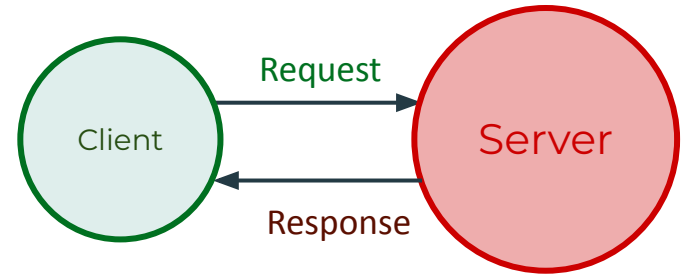
GET: request resource

Client

Architecture Patterns - Client-Server Pattern

API - Application Programming Interface:

API stands for **Application Programming Interface**, which is a set of definitions and protocols for building and integrating application software. **APIs** let your **product** or **service communicate** with **other products** and **services** without having to know how they're implemented.



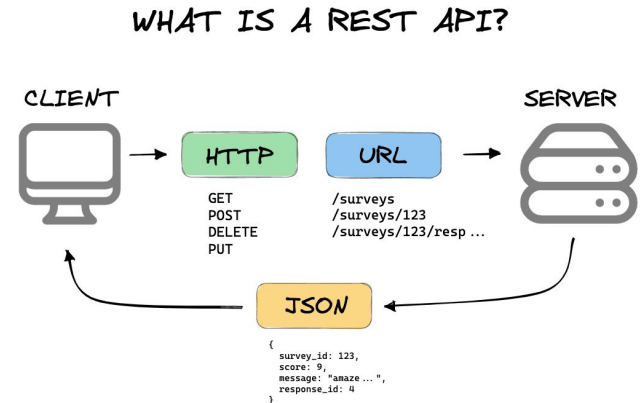
Architecture Patterns - Client-Server Pattern

RESTful API :

A **REST API** (a.k.a **RESTful API**) is an API that conforms to the constraints of **REST** architectural style. **REST** stands for **RE**presentational **S**tate **T**ransfer and is a set of architectural constraints, not a protocol or a standard.

In order for an API to be considered **RESTful**, it has to conform to these criteria:

- Client-Server Architecture.
- Statelessness.
- Cacheability.
- Layered system.
- Uniform Interface.
- (Often) **HTTP** Based.



<https://mannhowie.com/rest-api>

https://ics.uci.edu/~fielding/pubs/dissertation/fielding_dissertation.pdf

<https://www.redhat.com/en/topics/api/what-are-application-programming-interfaces>

Architecture Patterns - Client-Server Pattern

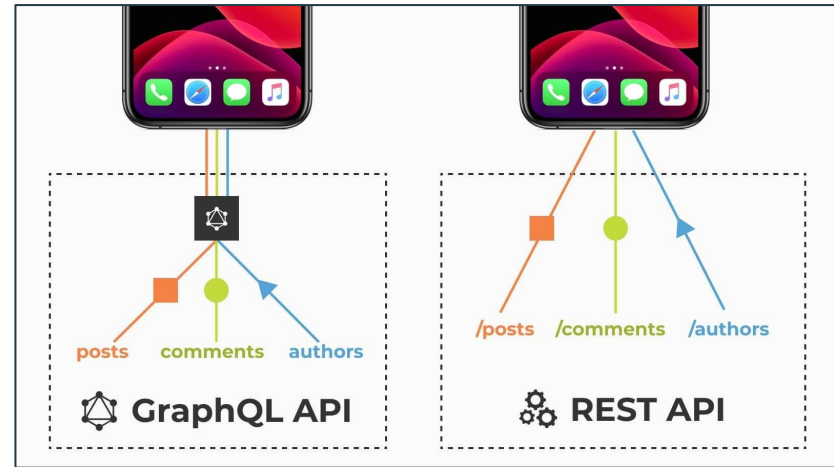
Note - REST API and GraphQL:

REST API is not the only way to implement APIs.

GraphQL is an open-source data query and manipulation language for APIs and a query runtime engine.

GraphQL enables declarative data fetching where a client can **specify exactly what data it needs from an API**. Instead of multiple endpoints that return separate data, a GraphQL server exposes a single endpoint and responds with precisely the data a client asked for.

<https://en.wikipedia.org/wiki/GraphQL>



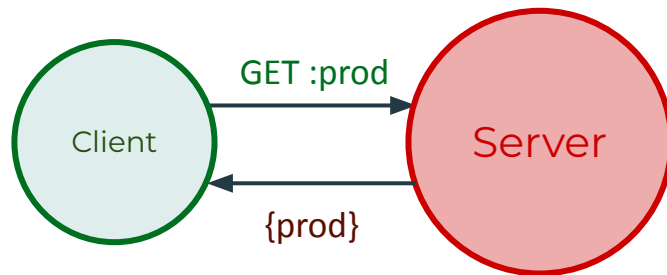
<https://www.mobilelive.ca/blog/graphql-vs-rest-what-you-didnt-know>

Section 7 - More components

Cloud and Architecture - Part III -
Adding a database in our Architecture

Persistence

Once our server is shutdown, all the products are **lost**. Our system is **not persistent !**

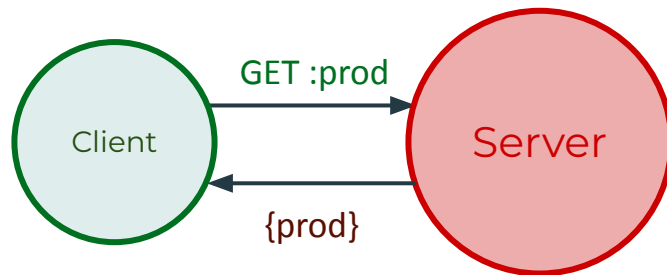


Persistence

Once our server is shutdown, all the products are **lost**. Our system is **not persistent !**

Persistence refers to the characteristic of state of a system that outlives (persists more than) the process that created it. This is achieved in practice by storing the state as data in **computer data storage**.

[https://en.wikipedia.org/wiki/Persistence_\(computer_science\)](https://en.wikipedia.org/wiki/Persistence_(computer_science))



Persistence

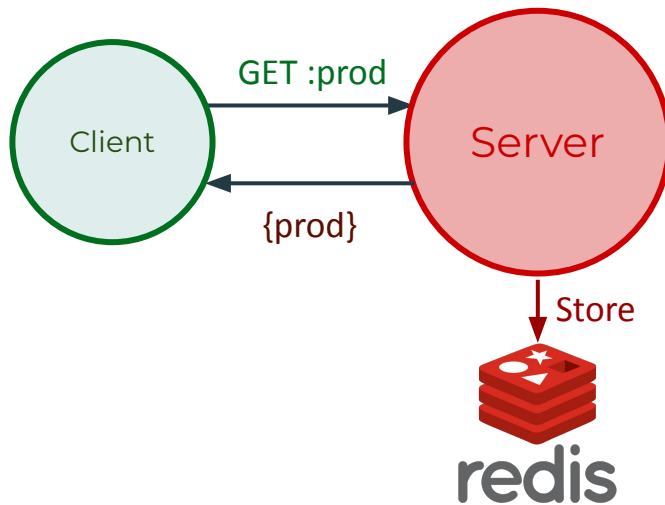
Once our server is shutdown, all the products are **lost**. Our system is **not persistent** !

Persistence refers to the characteristic of state of a system that outlives (persists more than) the process that created it. This is achieved in practice by storing the state as data in **computer data storage**.

[https://en.wikipedia.org/wiki/Persistence_\(computer_science\)](https://en.wikipedia.org/wiki/Persistence_(computer_science))

Exercise 2:

Make your system **persistent** using **redis** **key-value store (NoSQL Database)**.



Persistence

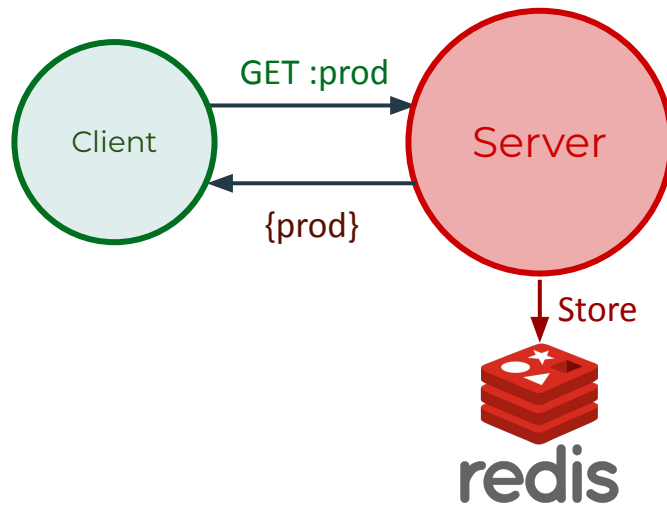
Once our server is shutdown, all the products are **lost**. Our system is **not persistent** !

Persistence refers to the characteristic of state of a system that outlives (persists more than) the process that created it. This is achieved in practice by storing the state as data in **computer data storage**.

[https://en.wikipedia.org/wiki/Persistence_\(computer_science\)](https://en.wikipedia.org/wiki/Persistence_(computer_science))

Exercise 2:

Make your system **persistent** using **redis** **key-value store (NoSQL Database)**.



Questions:

- What is a NoSQL Database ?
- Enumerate and define at least 4 different kind of databases.
- What is a hosted database ?

Section 8 - Solution

Cloud and Architecture - Part III -
Adding a database in our Architecture

Architecture Patterns - Using Redis

Python Server

```
curl -X 'POST' 'http://localhost:8000/products/' \
-H 'Content-Type: application/json' \
-d '{"name": "milk", "price": 4, "quantity": 12}'
```

POST: Create resource

```
curl -X 'GET' 'http://localhost:8000/products' \
-H 'Accept: application/json'
```

JSON

```
[{
  "name": "milk",
  "price": 4.0,
  "quantity": 12
}]
```

GET: request resource

Client

NoSQL Databases

NoSQL, also referred to as “not only SQL”, “non-SQL”, is an approach to database design that enables the **storage and querying of data outside the traditional structures** found in relational databases.

NoSQL Databases

NoSQL, also referred to as “not only SQL”, “non-SQL”, is an approach to database design that enables the **storage and querying of data outside the traditional structures** found in relational databases.

Instead of the **typical tabular structure of a relational database**, NoSQL databases, house data within **one data structure, such as JSON document**. Since this non-relational database design **does not require a schema (less stringent constraints)**, it offers rapid scalability to manage large and typically **unstructured (semi-structured) data sets**.

*<https://www.ibm.com/topics/nosql-databases>

NoSQL Databases

NoSQL, also referred to as “not only SQL”, “non-SQL”, is an approach to database design that enables the **storage and querying of data outside the traditional structures** found in relational databases.

Instead of the **typical tabular structure of a relational database**, NoSQL databases, house data within **one data structure, such as JSON document**. Since this non-relational database design **does not require a schema (less stringent constraints)**, it offers rapid scalability to manage large and typically **unstructured (semi-structured) data sets**.



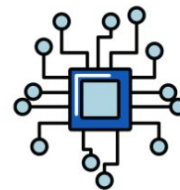
Schema-agnostic

No up-front schema design required during application development



Non-relational

No forcing of non-relational data into rows and columns



Inherently distributable

Distributed on commodity hardware for high availability & large-scale data management

Main Characteristics of NoSQL Databases

*<https://thedigitalskye.com/2021/03/09/7-must-know-ideas-about-nosql/>

*<https://www.ibm.com/topics/nosql-databases>

NoSQL Databases - Examples

Key-value database

- Deliver web advertisements
- Store & retrieve user session data
- High-speed in-memory data caching

Document database

- Publish digital content (Content Management System)
- Manage unstructured data feeds
- Join data streams



Column-oriented database

- Store sparse data (i.e. columns don't have a value)
- Record & analyze log files
- Build data summaries

Graph database

- Build a web of facts
- Reconstruct processes
- Manage social graphs (e.g. social networks, organized crime detection)

NoSQL Databases - Examples

Key-value database

Deliver web advertisements
Store & retrieve user session data
High-speed in-memory data
caching

E.g. **Redis**

Document database

Publish digital content (Content
Management System)
Manage unstructured data feeds
Join data streams

E.g. **MongoDB**



Column-oriented database

Store sparse data (i.e. columns
don't have a value)
Record & analyze log files
Build data summaries

E.g. **Cassandra**

Graph database

Build a web of facts
Reconstruct processes
Manage social graphs (e.g. social
networks, organized crime detection)

E.g. **Neo4J**

<https://thedigitalskye.com/2021/03/09/7-must-know-ideas-about-nosql>

Hosted Databases

In database hosting, **a third party** offers the **hardware** and **infrastructure** to run a database of the client's choosing, often in the cloud. They also configure the environment for **secure access**, **ensure resources are available to scale** the database as needed, and offer **managed services** based on requirements.

*<https://www.mongodb.com/database-hosting>

Hosted Databases

In database hosting, a **third party** offers the **hardware** and **infrastructure** to run a database of the client's choosing, often in the cloud. They also configure the environment for **secure access**, **ensure resources are available to scale** the database as needed, and offer **managed services** based on requirements.

*<https://www.mongodb.com/database-hosting>

Database as a service (DBaaS): the provider maintains the system infrastructure and database and delivers it as a **fully managed cloud service**. The service covers high-level administrative functions, such as **database installation**, **configuration**, **maintenance** and **upgrades**. Additional tasks, such as **backups**, **patching**, **performance management** and **security**, typically are also handled by the provider.

*<https://www.techtarget.com/searchdatamanagement/definition/database-as-a-service-DBaaS>

Hosted Databases

In database hosting, a **third party** offers the **hardware** and **infrastructure** to run a database of the client's choosing, often in the cloud. They also configure the environment for **secure access**, **ensure resources are available to scale** the database as needed, and offer **managed services** based on requirements.

*<https://www.mongodb.com/database-hosting>

Database as a service (DBaaS): the provider maintains the system infrastructure and database and delivers it as a **fully managed cloud service**. The service covers high-level administrative functions, such as **database installation**, **configuration**, **maintenance** and **upgrades**. Additional tasks, such as **backups**, **patching**, **performance management** and **security**, typically are also handled by the provider.

*<https://www.techtarget.com/searchdatamanagement/definition/database-as-a-service-DBaaS>



E.g. Oracle (Relational/SQL)

Hosted Databases

In database hosting, a **third party** offers the **hardware** and **infrastructure** to run a database of the client's choosing, often in the cloud. They also configure the environment for **secure access**, **ensure resources are available to scale** the database as needed, and offer **managed services** based on requirements.

*<https://www.mongodb.com/database-hosting>

Database as a service (DBaaS): the provider maintains the system infrastructure and database and delivers it as a **fully managed cloud service**. The service covers high-level administrative functions, such as **database installation**, **configuration**, **maintenance** and **upgrades**. Additional tasks, such as **backups**, **patching**, **performance management** and **security**, typically are also handled by the provider.

*<https://www.techtarget.com/searchdatamanagement/definition/database-as-a-service-DBaaS>



E.g. Oracle (Relational/SQL)



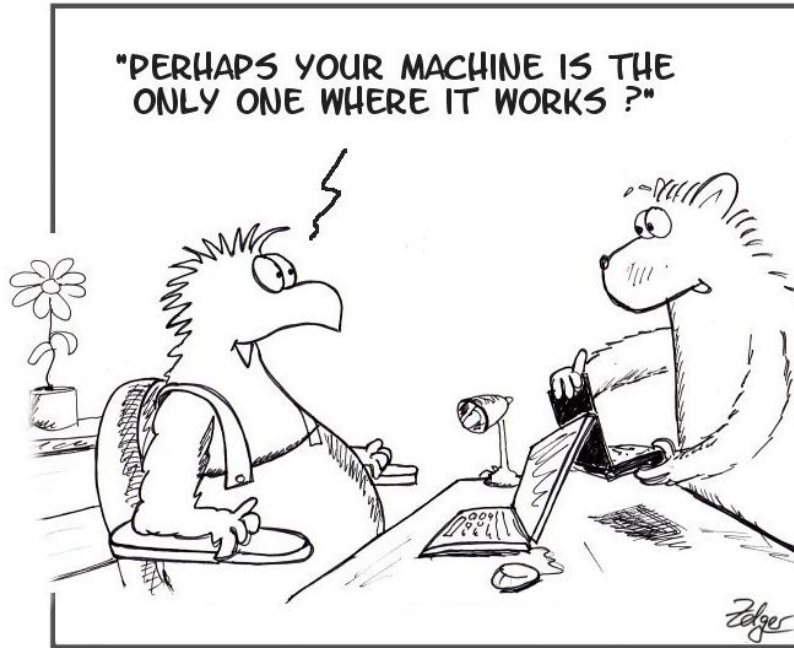
mongoDB

E.g. MongoDB (Document/NoSQL)

Section 9 - Portability

Well, try to run the code of your teammate !

Portability - Quick Overview



It works on my machine

Portability - Quick Overview

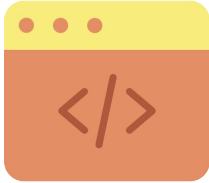
Porting is the task of doing any work necessary to make the computer program run in the new environment. Likewise, the term portability itself is a measure of whether software can be ported **without requiring major rework**.

Portability - Quick Overview

Porting is the task of doing any work necessary to make the computer program run in the new environment. Likewise, the term portability itself is a measure of whether software can be ported **without requiring major rework**.

For instance, a portable application could work on platforms such as Windows, Linux, iOS and Android. The porting process might include transferring installed program files to another system or creating the same program using different source code.

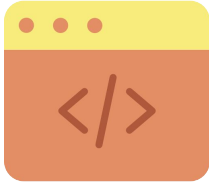
Portability - Types



Source code portability

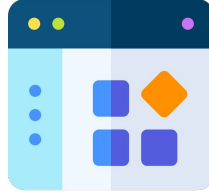
refers to building a program in a way that the same source code works in different environments. However, all the platforms must support the same programming language.

Portability - Types



Source code portability

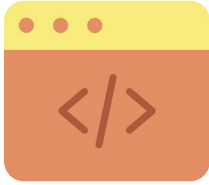
refers to building a program in a way that the same source code works in different environments. However, all the platforms must support the same programming language.



Application/SW portability

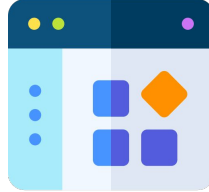
refers to softwares that are designed with portability in mind. Portable applications can be executed from one environment to another.

Portability - Types



Source code portability

refers to building a program in a way that the same source code works in different environments. However, all the platforms must support the same programming language.



Application/SW portability

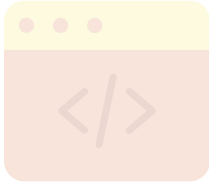
refers to softwares that are designed with portability in mind. Portable applications can be executed from one environment to another.



Data portability

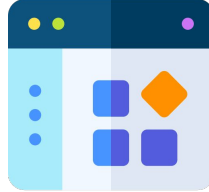
refers to data that's movable from one database or data center to another. If the data isn't readily movable to another environment, it must be re-created for the other system.

Portability - Types



Source code portability

refers to building a program in a way that the same source code works in different environments. However, all the platforms must support the same programming language.



Application/SW portability

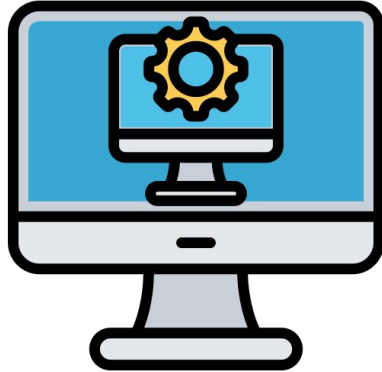
refers to softwares that are designed with portability in mind. Portable applications can be executed from one environment to another.



Data portability

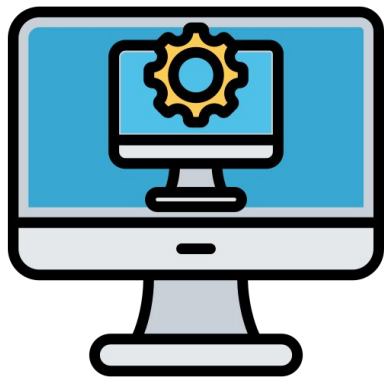
refers to data that's movable from one database or data center to another. If the data isn't readily movable to another environment, it must be re-created for the other system.

Virtualization



Virtualization is a process that allows for **more efficient utilization of physical computer hardware** and is the **foundation of cloud computing**.

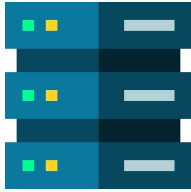
Virtualization



Virtualization is a process that allows for **more efficient utilization of physical computer hardware** and is the **foundation of cloud computing**.

Virtualization uses software to create an **abstraction layer** over computer hardware that allows the hardware elements of a single computer—processors, memory, storage and more—to be **divided into multiple virtual computers**, commonly called virtual machines (VMs). Each VM runs **its own operating system (OS)** and **behaves like an independent computer**, even though it is running on just a portion of the actual underlying computer hardware.

Virtualization Benefits (1/2)



1. Resource Efficiency

Server virtualization lets you run several applications —each on its own VM with its own OS— on a single physical computer without sacrificing reliability. This enables maximum utilization of the physical hardware's computing capacity.

Virtualization Benefits (1/2)



1. Resource Efficiency

Server virtualization lets you run several applications —each on its own VM with its own OS— on a single physical computer without sacrificing reliability. This enables maximum utilization of the physical hardware's computing capacity.



2. Easier management

Replacing physical computers with software-defined VMs makes it easier to use and manage policies written in software. This allows you to create automated IT service management workflows. (e.g. automated deployment)

Virtualization Benefits (2/2)



3. Minimal Downtime

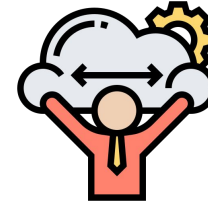
OS and application crashes can cause downtime and disrupt user productivity. Admins can run multiple redundant virtual machines alongside each other and failover between them when problems arise. Running multiple redundant physical servers is more expensive.

Virtualization Benefits (2/2)



3. Minimal Downtime

OS and application crashes can cause downtime and disrupt user productivity. Admins can run multiple redundant virtual machines alongside each other and failover between them when problems arise. Running multiple redundant physical servers is more expensive.

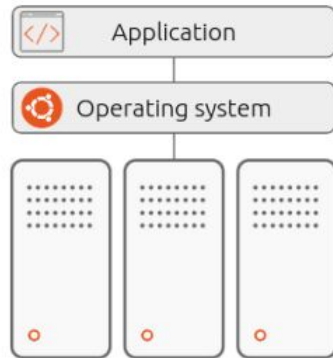


4. Faster Provisioning

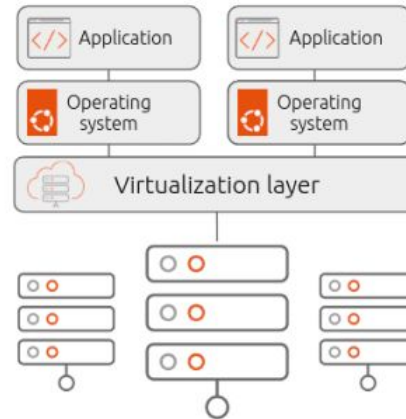
Buying, installing, and configuring hardware for each application is time-consuming. Provided that the hardware is already in place, provisioning virtual machines to run all your applications is significantly faster (can be automated).

Types of Virtualization - in a nutshell

There are several ways of categorizing types of virtualization. The sheer amount of existing virtualization schemes and due to the nature of this course, we present here a short overview of existing types of virtualization.



Traditional environment



Virtualized environment

<https://ubuntu.com/blog/containerization-vs-virtualization>

Types of Virtualization - in a nutshell

There are several ways of categorizing types of virtualization. The sheer amount of existing virtualization schemes and due to the nature of this course, we present here a short overview of existing types of virtualization.

1. **Hardware Virtualization :** The main job of hypervisor (in HW virtualization) is to control and monitor the processor, memory and other hardware resources.
2. **Operating System (OS) Virtualization :** In this kind of virtualization, the virtualization software is installed over the OS, and further use creates several other VMs (isolated user spaces).
3. **Data Virtualization :** Data virtualization tools create a software layer between the applications accessing the data and the systems storing it.

...Others: Network, Storage, Desktop, GPU, Application (e.g. JVM).

Hardware Virtualization

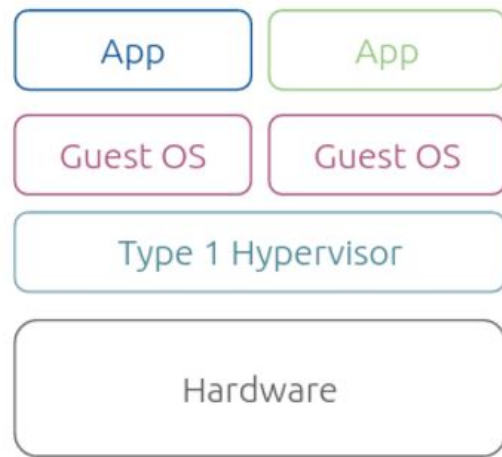
In a traditional physical computing environment, software such as an operating system (OS) or enterprise application has **direct access to the underlying computer hardware and components**, including the processor, memory, storage, certain chipsets and OS driver versions.

Hardware Virtualization

In a traditional physical computing environment, software such as an operating system (OS) or enterprise application has **direct access to the underlying computer hardware and components**, including the processor, memory, storage, certain chipsets and OS driver versions.

Hardware virtualization installs a hypervisor or virtual machine manager (VMM), which creates an **abstraction layer between the software and the underlying hardware**.

Once a **hypervisor** is in place, **software relies on virtual representations of the computing components**, such as virtual processors rather than physical processors. Popular hypervisors include VMware's vSphere, based on ESXi, and Microsoft's Hyper-V.



Type I Virtualization

OS Virtualization

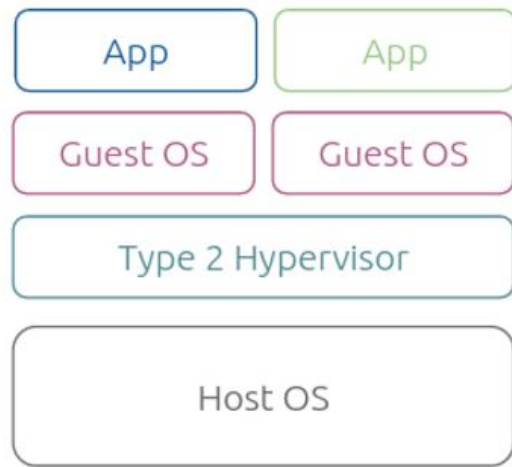
Unlike hardware-based virtualization, **OS virtualization is done over the top of the operating system**. This implies that the software installed on the OS virtualizes the same, making it the host.

OS Virtualization

Unlike hardware-based virtualization, **OS virtualization is done over the top of the operating system**. This implies that the software installed on the OS virtualizes the same, making it the host.

In this kind of virtualization, the virtualization software (VMM) is installed over the OS, and further **allows to the existence of multiple isolated VM..** Such instances **may look like real computers from the point of view** of programs running in them. We distinguish:

1. **Type II Virtualization:** Less efficient than Type I Virtualization but more handy to use. We cite VMware workstation and Oracle VirtualBox.



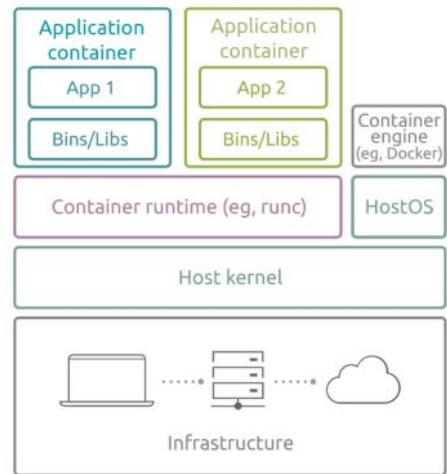
Type II Virtualization

OS Virtualization

Unlike hardware-based virtualization, **OS virtualization is done over the top of the operating system**. This implies that the software installed on the OS virtualizes the same, making it the host.

In this kind of virtualization, the virtualization software (VMM) is installed over the OS, and further **allows to the existence of multiple isolated VM..** Such instances **may look like real computers from the point of view** of programs running in them. We distinguish:

1. **Type II Virtualization:** Less efficient than Type I Virtualization but more handy to use. We cite VMware workstation and Oracle VirtualBox.
2. **Containerization:** does not need an intermediate hypervisor and can rely directly on the host OS. We cite Docker (but there is a **jungle** of tools).



Application containers (eg, Docker)

Containerization

Data Virtualization

Modern enterprises store data from **multiple applications**, using multiple file formats, in multiple locations, **ranging from the cloud to on-premise hardware and software systems**.

Data virtualization lets any application access all of that data—**irrespective of source, format, or location**.

Data Virtualization

Modern enterprises store data from **multiple applications**, using multiple file formats, in multiple locations, **ranging from the cloud to on-premise hardware and software systems**.

Data virtualization lets any application access all of that data—**irrespective of source, format, or location**.

It is a method for **combining data** from various sources and different types **into a comprehensive, logical representation** without physically relocating the data.

Simply put, users **can theoretically access and examine data while it still exists in its original sources thanks to dedicated middleware**. For instance Data Lake is a concept used to describe a form of such virtualization in contrast to Data Silos.

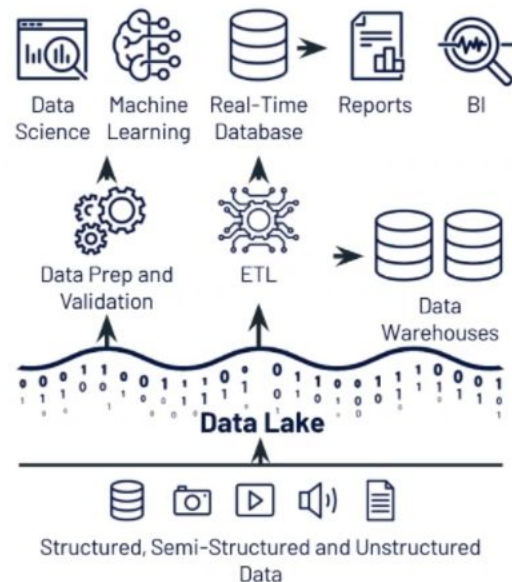
Data Virtualization

Modern enterprises store data from **multiple applications**, using multiple file formats, in multiple locations, **ranging from the cloud to on-premise hardware and software systems**.

Data virtualization lets any application access all of that data—**irrespective of source, format, or location**.

It is a method for **combining data** from various sources and different types **into a comprehensive, logical representation** without physically relocating the data.

Simply put, users **can theoretically access and examine data while it still exists in its original sources thanks to dedicated middleware**. For instance Data Lake is a concept used to describe a form of such virtualization in contrast to Data Silos.



Playing with Virtualization

Exercise 3:

Make your system portable (**Exercise 1**) using virtualization (**Use Docker**)

Questions:

What is Docker ?

Steps:

- Replace your installed redis by a **redis docker**.
- Write a **Dockerfile** to define your **docker image**.
- Run your docker image.
- Share your docker image with your teammate using hub.docker.com